NEXTGEN QA

NextGen QA

An AI-powered system that transforms Agile conversations into comprehensive, self-healing test suites



Project Supervision

Supervisor



Ms.Suriyaa Kumari

Senior Lecturer

Co-Supervisor



Mr.Eishan Weerasinghe

Assistant Lecturer

NEXTGEN QA - VOICE DRIVEN TEST CASE GENERATION AND SELF-HEALING

Component 1 – AI driven User story Generation

- Capture the Agile meeting discussion
- Transcribe them and convert convercation into user stories

Component 2 – Intelligent Test Case Generation

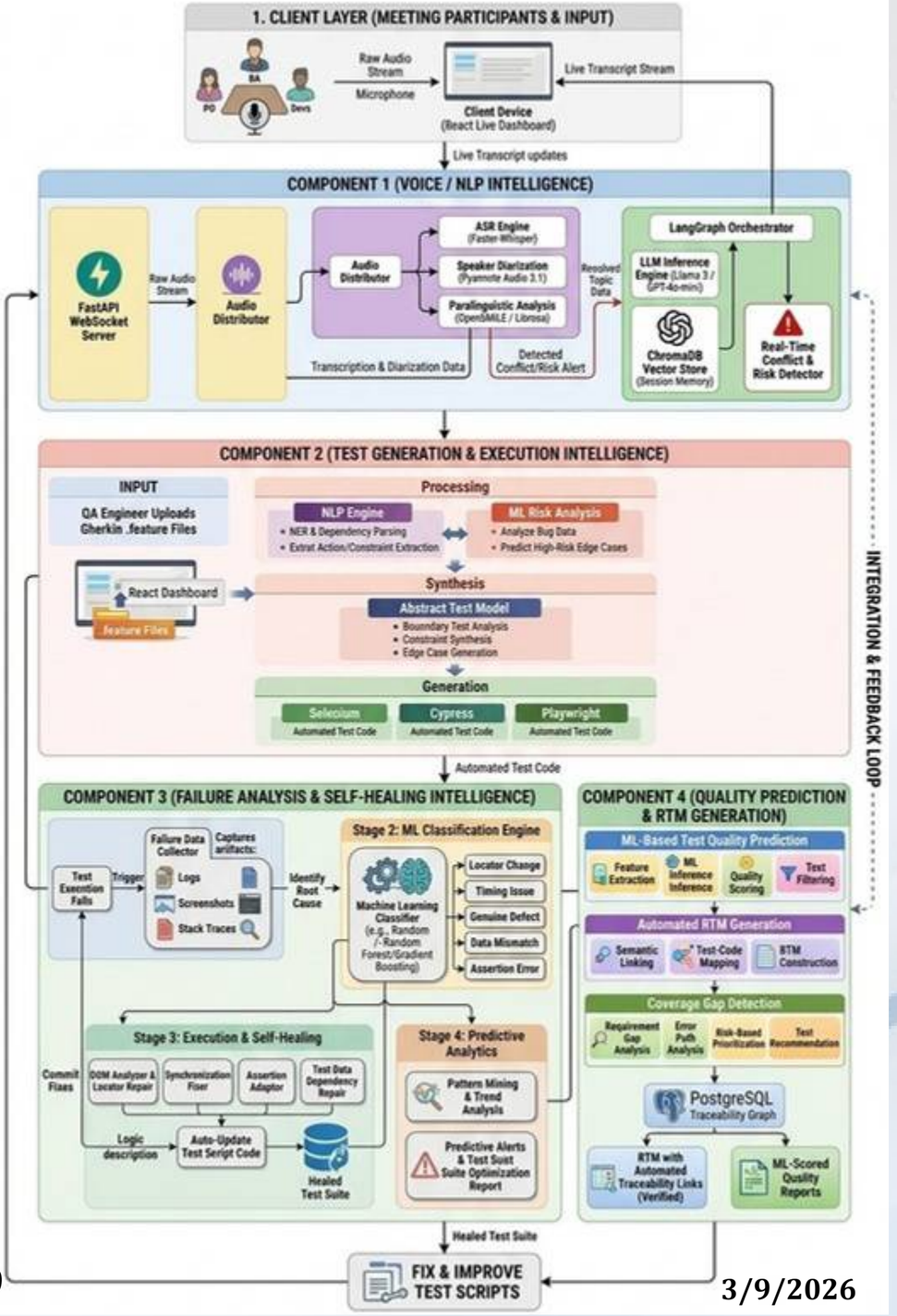
- Automatically convert user stories into structured test scenario
- Generate test script for testing frameworks

Component 3 – Failure Analysis and Self-Healing System

- Analysis automated test failure and identify root cause
- automatically repir test script

Component 4 – Test Quality Prediction & Automated RTM

- Eveluate qulity of generated test case
- Maintance RTM Automatically



NEXTGEN QA - VOICE DRIVEN TEST CASE GENERATION AND SELF-HEALING

THE PROBLEM

- Manual requirement capture in Agile meetings causes loss of context and incomplete information.
- Manual test case creation is time-consuming and may miss important edge cases.
- Frequent automated test failures in CI/CD require manual debugging and script fixing.
- Weak test validation and poor traceability lead to gaps between requirements, tests, and code.

IMPACT OF THE PROBLEM TO USERS AND SOCIETY

- ✔ Slower software delivery
- ✔ Reduced software quality
- ✔ Higher development costs
- ✔ Poor user experience

USERS AND STAKEHOLDERS











NEXTGEN QA - VOICE DRIVEN TEST CASE GENERATION AND SELF-HEALING

Existing Solutions and Their Limitations

- **Otter.ai:** Only convert speech to text without real time conflict detect
- **Copilot:** Require human prompt for different framework
- **Selenium:** Do not automatically repair broken test scripts
- **Jira Xray:** RTM maintenance and linking are manual

Solution Overview and how it overcomes the limitations

- Uses AI to converts voice discussions into structured user stories automatically.
- The system generates test cases using AI and analyzes test failures using machine learning.
- Self-healing mechanism fix broken test scripts automatically and maintains RTM automatically.

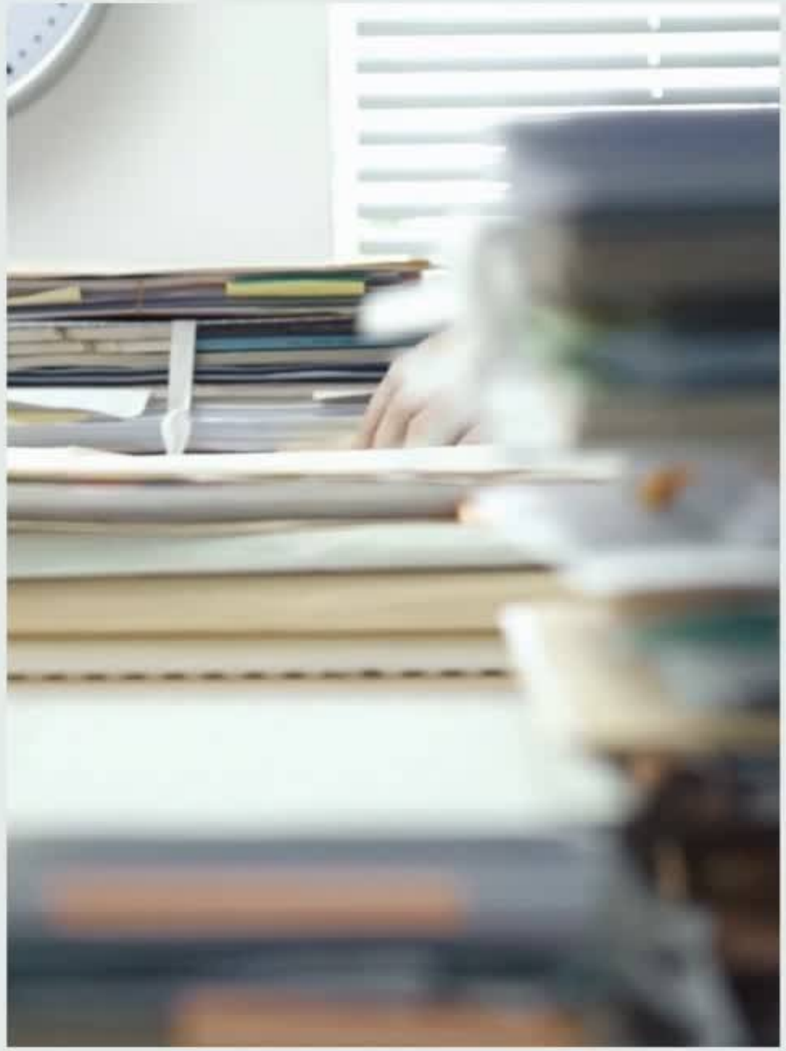
The Proposed Solution

- Converts voice discussions into structured user stories and detects conflicts.
- Automatically generates multi-framework test cases using AI
- Detects and analyzes test failures using ML
- Applies self-healing mechanisms to repair unstable test scripts.
- Predicts test quality and maintains the automated RTM

Individual Student Components

- Component 1 – AI driven User story Generation
- Component 2 – Intelligent Test Case Generation
- Component 3 – Failure Analysis and Self-Healing System
- Component 4 – Test Quality Prediction & Automated RTM

NEXTGEN QA - VOICE DRIVEN TEST CASE GENERATION AND SELF-HEALING



Manual test case creation

IT22251114 | Jinad Induvidwa A.G



AI driven User story Generation

Contribution to the Overall Solution

- **Identifies speakers and transcribes** in multi-speaker meetings in real time.
- Converts messy live speech into structured, logical **JSON user stories.**
- Automatically **scores generated stories for testability using INVEST criteria.**

Novelty

The system detects conflicts in the requirements, it immediately shows as a Red Warning .

Problem Definition

- Manual requirement capture some information and context can be lost.
- Existing tools only convert speech to text.
- Transcription and requirement validation are separate and slow processes.



Novelty

- Introduces "**Visual Warning Protocol**" (Red Warning)
- Simultaneously merges **diarization**, **risk scoring**, and **conflict detection**

Research Objectives

- **Main**: **Real-time AI System to transcribe meetings**, **detect conflicts**, and **generate JSON user stories**.
- **Specific**:
  - Assign "Acoustic Risk Scores" from voice cues.
  - Detect requirement conflicts within 5 seconds.
  - Automatically generates structured JSON user stories and checks their quality

Applied Knowledge Domains & Approach

- Follow **Design Science Research (DSR)** approach to Solve losing important information during meetings.
- Requirement Engineering, Conflict Resolution, Real-Time Tech

Technological Implementation in the Domain

  - **WebRTC** - capturing audio
  - **Faster-Whisper** - speech recognition
  - **Pyannote** - identifying speakers.
- **Llama 3** and **ChromaDB** - analyze conversation and detect requirement conflicts in real time.











Validation & Evaluation Strategy

Targets performance metrics:

  - **Real-Time Factor < 1.0,**
  - **Word Error Rate < 15%**
  - **Conflict detection within a 5-second window.**
  - **Validate the generated user stories using INVEST criteria**

Data Security & Ethical Clearance

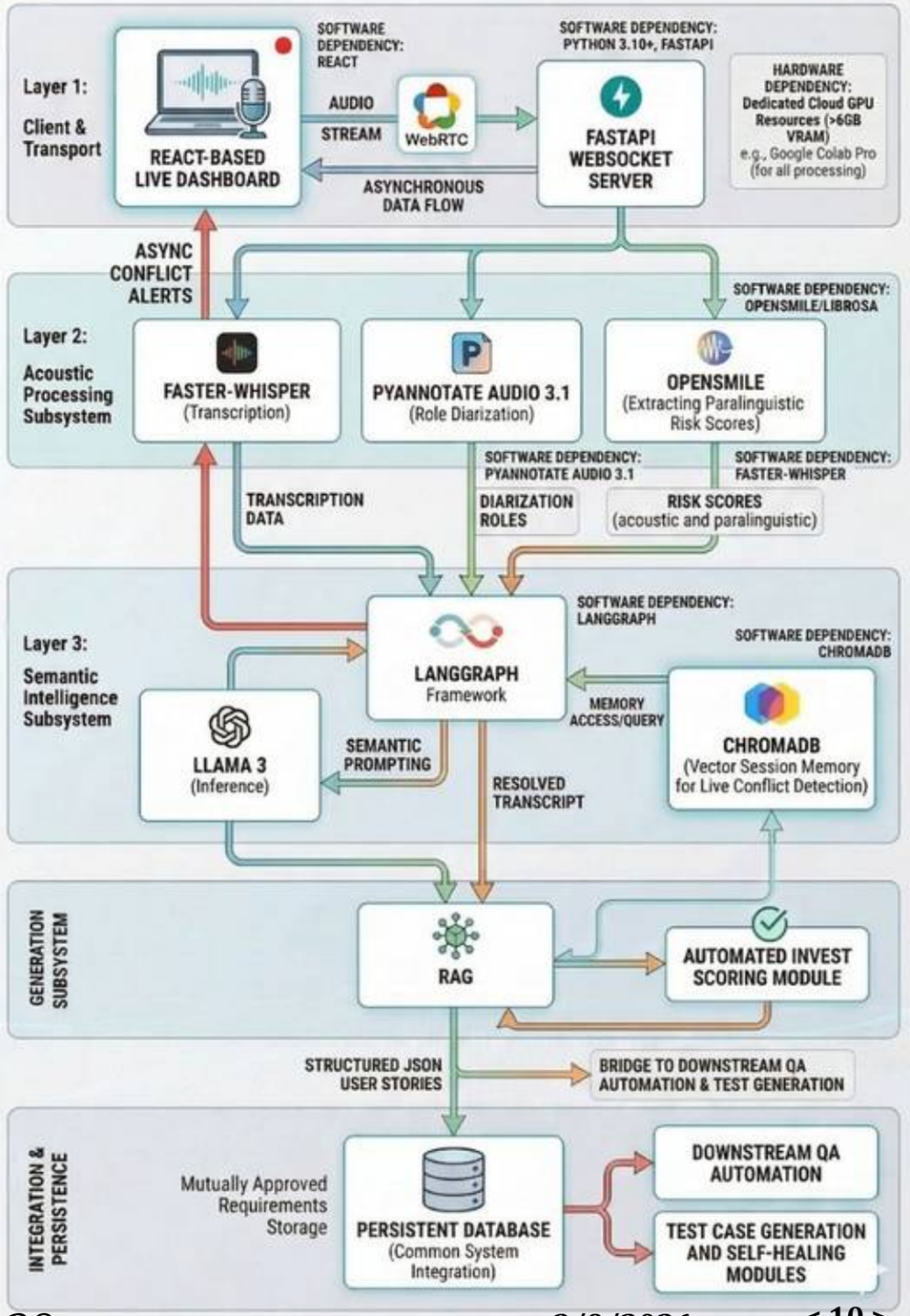
- Meeting audio and text are processed only in **temporary memory and then deleted.**
- Only finalized, team-approved **JSON User Stories are saved.**

System Overview

- **React dashboard:** captures meeting audio from multiple speakers.
- **Acoustic processing system:** converts speech to text, identifies speakers, and analyzes voice cues
- **Semantic intelligence system:** analyze conversation and detect requirement conflicts in real time.
- **Generation module:** converts the discussion into structured JSON user stories and checks them using INVEST criteria.
- Approved user stories are then saved in the DB

Dependencies

- **For software** : Python 3.10+, FastAPI, React, LangGraph, ChromaDB, Pyannote Audio, Faster-Whisper, and OpenSMILE.
- **For hardware:** Cloud GPU resources are required for f low-latency local inference.
- **For persistent :** JSON user stories are stored in a central database





Intelligent Test Case Generation with Multi-Framework Code Synthesis

Contribution to the Overall Solution

- Transforms user stories into structured Gherkin test scenarios automatically
- Generates comprehensive test cases including boundary values and edge cases
- Produces executable framework-specific test code (Selenium, Cypress, Playwright)
- Enables AI-assisted generation with human review and refinement capability

Novelty

- Human-in-the-loop approach
- Multi-framework code generation from single test scenario

Problem Definition

- Manual Effort
- Framework Fragmentation
- No Intelligent Automation

Novelty

- **Human-in-the-Loop Test Generation Architecture**
- **Multi-framework code generation from single test scenario**

Research Objectives

- Main :Develop an intelligent module that automatically transforms Gherkin acceptance criteria into comprehensive test scenarios and generates type-safe, multi-framework test code.
- **Specific:**
  - **Semantic Parsing & Constraint Synthesis:** NER (Named Entity Recognition)/dependency parsing + boundary value analysis for scenario generation
  - **ML Edge Case Prediction:** Random Forest classifier predicts failure scenarios from bug history
  - **Multi-Framework Code Generation:** Abstract test model generates type-safe code for Selenium, Cypress, Playwright



Applied Knowledge Domains & Approach

- Uses **Design Science Research (DSR)** to iteratively design and refine an AI-assisted test generation system.
- Combines Software Testing, BDD, NLP, and ML for intelligent scenario generation and code synthesis..

Technological Implementation in the Domain

- Semantic parsing with **spaCy / NLP** extracts test elements from Gherkin scenarios.
- ML Module: Random Forest predicts risky edge cases from historical defect data.
- Generates type-safe scripts for Selenium, Cypress, and Playwright.

Validation & Evaluation Strategy

- QA experts review AI-generated tests, and all outputs require human approval before use.

  Test Generation Time - **< 10 seconds per scenario**

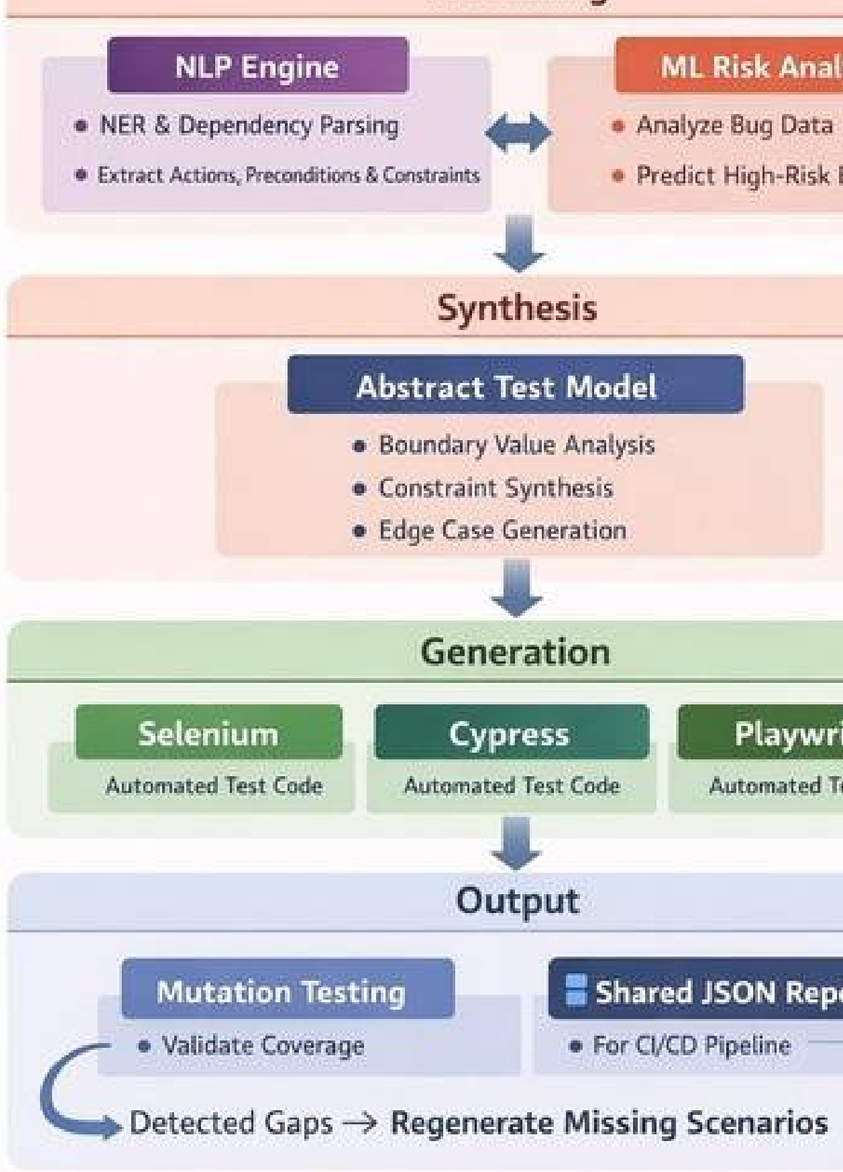
  Code Executability Rate **≥ 85% run without syntax errors**

Data Security & Ethical Clearance

- Uses only project test files and bug data, with no personal data involved.
- Test data and artifacts are secured with role-based access control and encrypted APIs.

System Overview of the Individual Component

- **Input Layer:**
  - React dashboard , voice-generated user stories
- **Semantic Processing:**
  - spaCy NER + boundary value analysis
- **ML Prediction:**
  - Random Forest edge-case prediction
- **Code Synthesis:**
  - Framework-agnostic type-safe code generation
  - Jinja2 templates and CodeLlama LLM
- **Validation:**
  - Mutation testing + coverage analysis



Dependencies & Common Components

- **Software:** Python, FastAPI, React, spaCy, scikit-learn, pytest/Jest/Playwright
- **Hardware:** Standard cloud compute; with GPU
- **Common Integration:** Approved test scenarios stored in database and executed through CI/CD



Failure Analysis & Self Healing

Contribution to the Overall Solution

- Collects test failure artifacts (logs, stack traces, execution data)
- Uses Machine Learning to identify root causes of failures
- Classifies failures such as locator changes, synchronization issues, environment errors, real application bugs
- Applies self-healing strategies to repair unstable test scripts
- Improves reliability of automated testing in CI/CD pipelines

Novelty

- ML-based **root cause analysis** of test failures
- Automated test script repair - **Self-healing**
- **Flaky test detection** using historical data

Problem Definition

- Automated tests in CI/CD pipelines frequently fail due to unstable test scripts and environment issues.
- Many failures are not real software defects but issues such as locator changes or synchronization problems.
- QA engineers must manually analyze logs and debug test failures
- This increases test maintenance effort and slows down software delivery



Novelty

- ML-based **root cause analysis** of automated test failures
- Automatic repair of broken test scripts- **Self Healing**
- **Flaky test detection** using historical execution data

Research Objectives

- **Main**: **Develop an ML-based system to automatically analyze test failures and perform self-healing in CI/CD pipelines.**
- **Specific**:
  - Collect test execution artifacts from CI/CD.
  - Classify failure root causes using ML.
  - Automatically repair unstable test scripts.
  - Detect flaky tests using historical data.

Research Methodology

- **Design Science Research (DSR)** approach is used. The system is designed, implemented, and evaluated to solve the automated test failure problem.

Knowledge Domains

🧪 **Software Testing & QA Automation** - Understanding automated test failures in CI/CD pipelines

🤖 **Machine Learning**

📄 **Log Analysis & Feature Engineering** - Extracting insights from logs and execution artifacts

⚙️ **DevOps & CI/CD Automation** - Integrating failure analysis into testing pipelines

Technologies Used

🐍 Python – system implementation

🧠 Scikit-learn – Random Forest / Gradient Boosting

🧪 Selenium / Playwright – automated test execution

🔄 GitHub Actions / Jenkins – CI/CD pipeline integration

Validation and Evaluation

📊 Predictive module evaluated by detecting flaky tests

📈 ML model evaluated by Accuracy, Precision, Recall, F1-score

🔧 Self-healing evaluated using healing success rate.

⏱️ Reduction in debugging time

Data Availability, Security & Ethics

🔒 Data from automated test logs only

🚫 No personal or sensitive data used

🎓 Used strictly for academic research

🗄️ Stored securely in research environment

System Overview of the Component

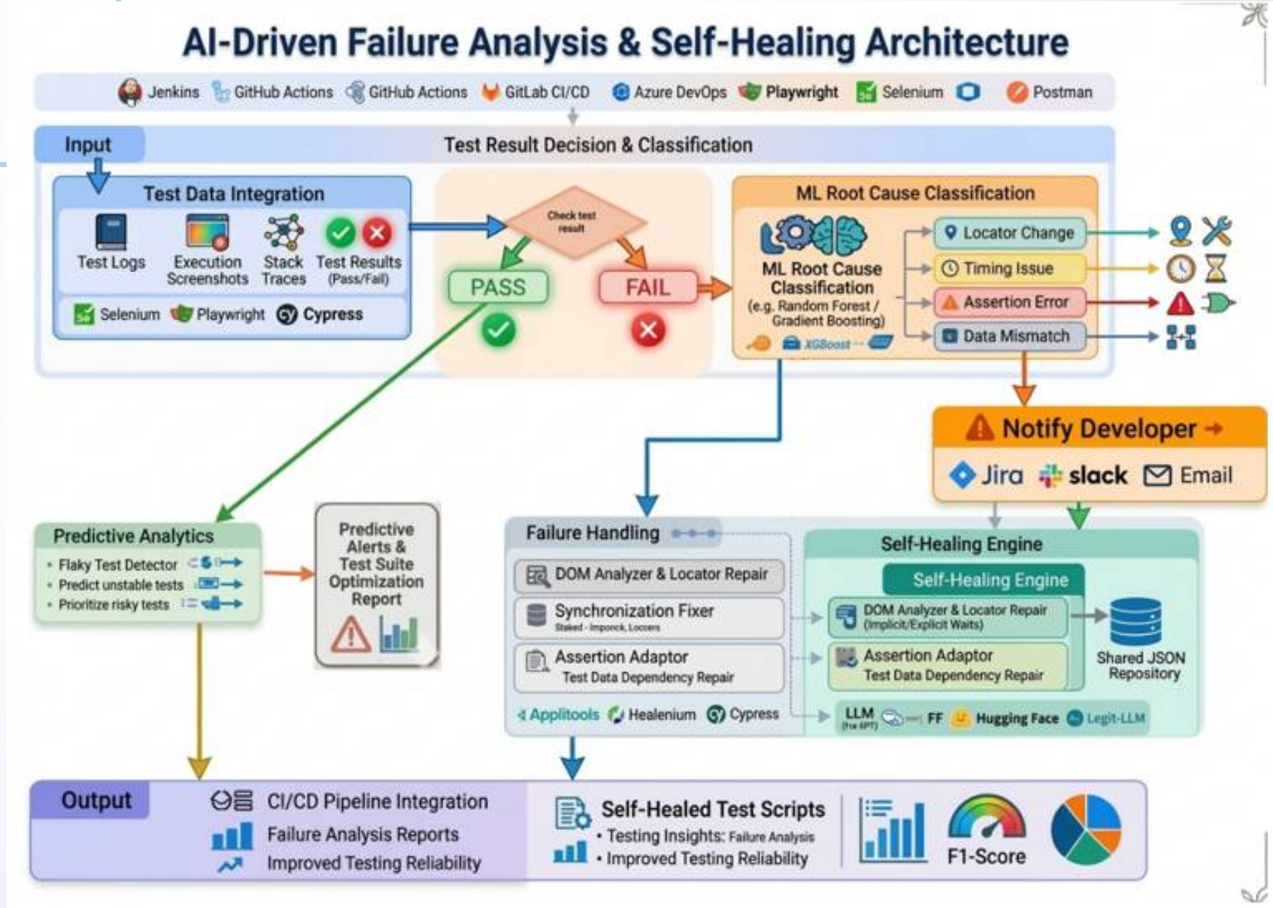
- Collects test failure artifacts from automated test executions (logs, stack traces, execution metadata)
- Performs feature extraction from failure data
- Uses Machine Learning models to classify root causes of failures
- Applies self-healing strategies (locator repair, synchronization fixes, retry mechanisms)
- Stores results for future learning and flaky test detection

Dependencies and Common Components

Input from Component 2 - Automated test execution results

Interaction with Component 4 -Provides failure insights for software quality prediction

For software: Python, Scikit-learn, Selenium, Pandas, NumPy, NLTK/spaCy, Jenkins or GitHub Actions.
For hardware: cloud environment to run automated tests and ML models.
For storage: PostgreSQL database to store test execution logs, failure artifacts, and historical data.





ML-BASED TEST QUALITY PREDICTION & AUTOMATED REQUIREMENTS TRACEABILITY MATRIX (RTM)

Contribution to the Overall Solution

- Predicts test case quality using machine learning before execution.
- Automatically generates an RTM linking requirements, test cases, and code modules.
- Identifies coverage gaps and redundant tests to improve testing efficiency.

Novelty

- Autonomous Requirements Traceability Framework
- Predictive Test Quality Intelligence Engine

Novelty

- **Autonomous RTM Framework** links requirements, tests, and code using semantic & coverage analysis.
- **Predictive Test Quality Engine** evaluates test case effectiveness before execution using machine learning.

Problem Definition

- Generated test cases often lack quality validation, leading to weak fault detection.
- RTM maintenance is manual and time-consuming, causing inconsistencies.
- Poor traceability between requirements, tests, and code results in coverage gaps.



Research Objectives

- **Main**: Develop an **ML-based QA component** to predict test case quality & generate an **automated RTM.**
- **Specific**:
  - Feature Extraction – Assertions, boundaries, error handling.
  - ML Scoring – Assigns 0–100 test quality score pre-execution.
  - Auto RTM Updates – Generates and maintains mappings.
  - Test Optimization – Detects gaps & redundant tests.

Applied Knowledge Domains & Approach

- Uses **Design Science Research (DSR)** to address weak test validation and manual traceability issues.
- Combines Software Engineering (test automation, traceability) with AI techniques (Machine Learning, semantic analysis).

Methodology

Transforms requirements, test cases, and code into an intelligent quality management platform.
Uses a structured process including data collection, ML model development, traceability generation, and optimization.

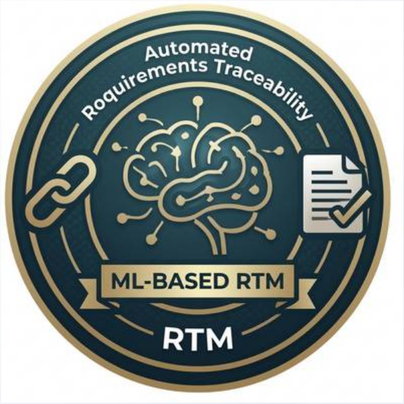


Technological Implementation in the Domain

- Uses **ML Model** & **feature extraction** to evaluate test quality (assertions, coverage, error handling).
- Uses **semantic similarity and coverage data** to automatically generate and maintain the **RTM**.

Validation & Evaluation Strategy

- Evaluates the ML model using **accuracy, precision, recall, and F1-score.**
- Measures system effectiveness through traceability coverage, gap detection, and test optimization.

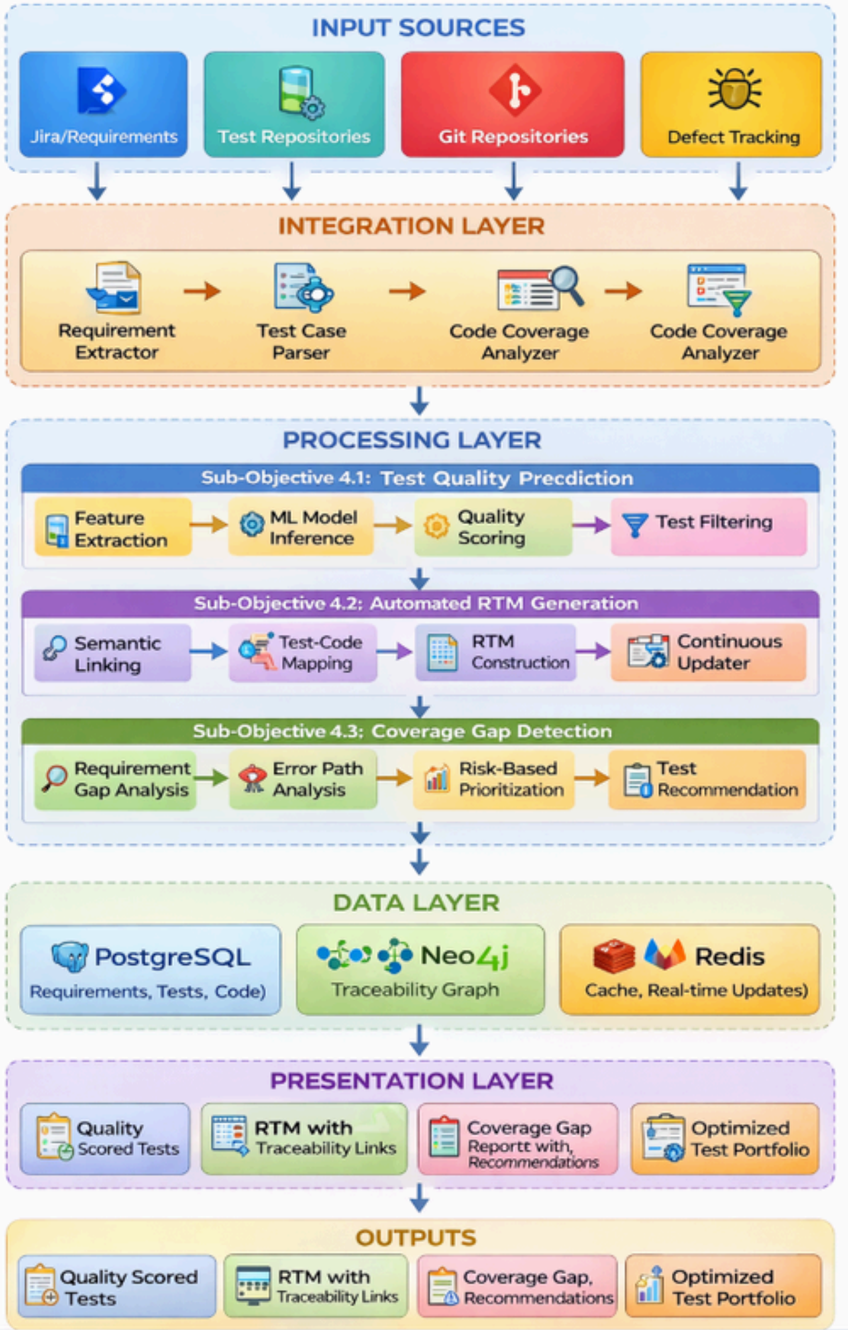


Data Security & Ethical Clearance

- Secure handling of requirements, test cases, and code metadata used for analysis.
- Ensures data confidentiality, storing only processed RTM mappings and test quality scores.

System Overview of the Individual Component

- **Input Sources Layer :** Collects data from Jira, Test Repositories, Git, and Defect Tracking systems.
- **Integration Layer :** Processes artifacts using Requirement Extractor, Test Parser, and Coverage Analyzer.
- **Processing Layer :** Performs ML-based test quality prediction, automated RTM generation, and coverage gap detection.
- **Data Layer:** Stores data in PostgreSQL, Neo4j, and Redis.
- **Presentation Layer:** Displays test quality scores, RTM links, and coverage reports.
- **Outputs**: Generates quality-scored tests, automated RTM, and optimized test recommendations.



BUSINESS POTENTIAL

Target Audience

- Software Testing Companies
- QA Engineers
- DevOps Teams
- Enterprise Development Teams

Free Version:

- Voice-driven test generation (Limited)
- Single-framework code generation (Selenium only)
- scenario generation
- Community support

Premium Version:

- Voice-driven test generation (Unlimited)
- Multi-framework code generation (Selenium, Cypress, Playwright)
- ML-based edge case prediction and Community support
- Mutation testing & coverage validation
- Self-healing locators

COMMERCIALIZATION STRATEGY

Customer Profile

- QA Engineers
- Software Test Automation Engineers
- DevOps Engineers
- Software Development Teams

Market Place

- Automates requirement understanding, test generation, and failure recovery using AI
- Multi-framework output - Selenium, Cypress, Playwright from one source
- Suitable for enterprise software teams practicing Agile and CI/CD

Target Market

- Software development companies
- QA automation teams
- DevOps-driven organizations
- Large enterprises with CI/CD pipelines

Investment / Cost

Development Cost

- AI model development and training
- Software engineering and system integration
- Cloud infrastructure for model execution

Operational Cost

- Cloud GPU/CPU resources
- Data storage and CI/CD infrastructure
- System maintenance and updates

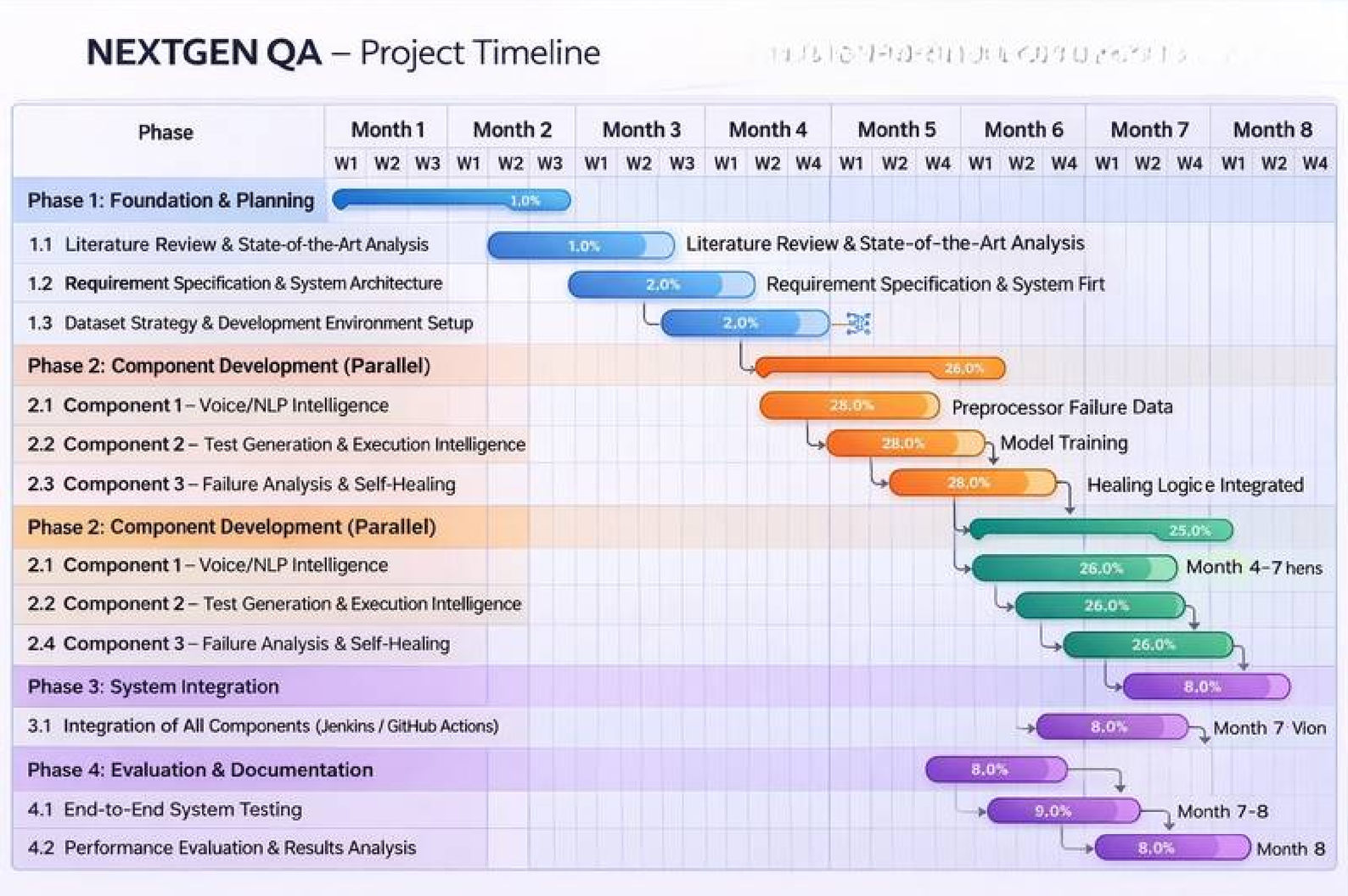
Value Proposition

- Faster test creation from requirements
- Intelligent failure diagnosis and automated repair
- Reduced testing maintenance cost
- Improved reliability of automated testing pipelines

Intellectual Property

- AI-based failure analysis and self-healing framework
- Automated requirement-to-test generation pipeline
- Integrated QA intelligence platform combining ML and DevOps

GANNT CHART


NEXTGEN QA – Project Timeline
Phase
Month 1
Month 2
Month 3
Month 4
Month 5
Month 6
Month 7
Month 8
W1 W2 W3 W1 W2 W3 W1 W2 W3 W1 W2 W4 W1 W2 W4 W1 W2 W4 W1 W2 W4 W1 W2 W4
Phase 1: Foundation & Planning
1.0%
1.1 Literature Review & State-of-the-Art Analysis
1.0%
Literature Review & State-of-the-Art Analysis
1.2 Requirement Specification & System Architecture
2.0%
Requirement Specification & System Firt
1.3 Dataset Strategy & Development Environment Setup
2.0%
Phase 2: Component Development (Parallel)
26.0%
2.1 Component 1 – Voice/NLP Intelligence
28.0%
Preprocessor Failure Data
2.2 Component 2 – Test Generation & Execution Intelligence
28.0%
Model Training
2.3 Component 3 – Failure Analysis & Self-Healing
28.0%
Healing Logic e Integrated
Phase 2: Component Development (Parallel)
25.0%
2.1 Component 1 – Voice/NLP Intelligence
26.0%
Month 4–7 hens
2.2 Component 2 – Test Generation & Execution Intelligence
26.0%
2.4 Component 3 – Failure Analysis & Self-Healing
26.0%
Phase 3: System Integration
8.0%
3.1 Integration of All Components (Jenkins / GitHub Actions)
8.0%
Month 7 Vion
Phase 4: Evaluation & Documentation
8.0%
4.1 End-to-End System Testing
9.0%
Month 7-8
4.2 Performance Evaluation & Results Analysis
8.0%
Month 8

WORK BREAKDOWN STRUCTURE

